

Semantic GPU Engine 第二世代 設計憲法 v0.1

0. この文書の目的

第二世代は、第一世代のコードを整理して作り直すプロジェクトではない。

第一世代で得られた経験をもとに、

「人間が目的を記述し、Compilerが実行方法を選択し、その判断を完全な実行成果物として固定する」

というGPU実行モデルを最初から設計する。

第一世代は凍結された教材・比較対象・D3D12挙動確認用実装として維持する。

第二世代は第一世代のソースコード、型、Visual Studioプロジェクト、Backend実装へ依存してはならない。

第1章 目的

第1条 最上位目的

このプロジェクトの目的は、既存ゲームエンジンの機能を再現することではない。

目的は、GPU実行に関する次の境界を再定義することである。

人間が指定する目的
Compilerが判断する手段
固定される実行成果物
実行時にのみ与えられる値
Backend固有の物理変換

人間は可能な限り目的と制約を書く。

Compilerは許された選択肢から実行方法を選ぶ。

BackendはCompilerの判断を再解釈せず、固定された成果物を対象APIへ写像する。

第2条 非目的

第二世代の初期段階では、次を目的としない。

既存ゲームエンジンとの機能数競争
最短期間でのゲーム完成

Vulkan対応
複数API対応
大量の描画機能
Editor
Asset pipeline
Scene graph
ECS
物理エンジン
製品向け最適化

これらはGPU実行抽象化の正しさを証明した後に検討する。

第2章 第一世代との関係

第3条 第一世代の固定

現在のrepoを第一世代参照実装として固定する。

第一世代へ加える変更は、原則として次に限定する。

重大なクラッシュ修正
資料として成立させるための文書修正
再現性維持のためのビルド修正
第二世代との比較に必要な計測追加

第一世代に新しい抽象化や大規模設計変更を追加しない。

第4条 コード移植禁止

第二世代へ第一世代の実装をコピーしてはならない。

移してよいものは次だけである。

要求仕様
不変条件
テストシナリオ
失敗例
観測結果
D3D12 Debug Layerから得た知見
アルゴリズムの数学的説明

既存関数を再利用したい場合は、先にその関数が満たしていた契約を文章化し、新しい型体系に対して再実装する。

第3章 五つの領域

第二世代は、情報を次の五領域へ分類する。

自由領域
意味領域
Compiler領域
固定領域
実行時領域

第一世代で使っていた四領域へ、動的実行入力を明示する「実行時領域」を追加する。

第5条 自由領域

自由領域は、アプリケーションまたは数学的理論が自然に持つ表現を所有する。

例：

Classical mesh
SDF
PGA
CGA
Voxel
Procedural geometry
Experiment scene
Camera model
Application-specific quality target

自由領域は、GPU実行方法を直接指定してはならない。

禁止される概念の例：

D3D12_RESOURCE_STATE
Descriptor heap
Root Signature
PSO
Command List
Fence
Queue ownership
Frame slot
Heap placement
Barrier

自由領域は、意味領域へLoweringするFrontendを提供する。

異なる自由領域同士は原則として依存しない。

共通する数学型や実験条件は、中立な共通プロジェクトへ置く。

第6条 意味領域

意味領域は、GPU上で成立させたい論理的事実を所有する。

意味領域が表現するもの：

どのResourceが存在するか
Resourceがどのような情報を表すか
どのWorkが存在するか
Workが何を読むか
Workが何を書くか
Programがどの意味的入力要求するか
Work間にどの時間関係があるか
どの出力を外部へ公開するか

意味領域は実行方法を決定しない。

意味領域に置いてはならないもの：

Direct / Compute / Copy queueの選択
PhysicalAllocationId
Frames in flight
Descriptor配置
Native state
PSO specialization
Shader target version
DXIL / DXBC
Heap種類
Fence値

意味領域は、D3D12を知らない。

意味領域は、Compilerが利用可能な選択肢を不必要に狭めない。

第7条 Compiler領域

Compiler領域は、意味を実行可能な形へ変換する判断を所有する。

Compilerが決めるもの：

正規化
妥当性検証
依存関係
実行順序
Queue割り当て
Resource寿命
Physical instance数
Memory allocation
Aliasing
Resource state
Subresource state
Queue synchronization
Frame boundary
Temporal relation
External acquire/release
Preparation
Shader target compile
Program binding layout
Executable specialization
Package operation order

Compilerは、Backendが判断しなくて済むところまで決定する。

Compiler内部の中間表現は外部ABIではない。

Compiler内部型をRuntimeまたはBackendへ公開してはならない。

第8条 固定領域

固定領域は、Compilerが決定した実行方法を完全に保存する。

中心成果物は `FrozenExecutablePackage` とする。

固定領域はSource IRを知らない。

固定領域はCompiler内部計画を知らない。

固定領域に推測を要求する欠落情報があってはならない。

固定領域は必ず次の性質を満たす。

同じPackage bytesは同じ意味を持つ
外部ファイル変更によって実行内容が変わらない
Compilerなしで読み込める
Sourceなしで再物質化できる
Device loss後もPackage-owned objectを再作成できる

Package単体を検証できる
安定したversionとdigestを持つ

第9条 実行時領域

実行時領域は、固定Packageに含めるべきでない動的値を所有する。

例：

Frame number
時間
Camera matrix
CPU更新Buffer内容
外部Texture
外部Buffer
Surface image
外部Resourceのincoming state
外部Resourceのoutgoing state
外部producer completion

実行時入力は `FrameInvocation` としてPackageとは別に渡す。

動的値の変更によってPackageを再コンパイルしてはならない。

構造が変わった場合のみ再コンパイルする。

第4章 依存方向

第10条 唯一の依存方向

許可される基本依存は次だけである。

自由領域
↓
意味領域
↓
Compiler領域
↓
固定領域
↓
Runtime契約
↓
Backend

実行時領域は固定Packageへ値を供給するが、Package定義を書き換えない。

第11条 禁止依存

次の依存は禁止する。

```
意味領域 → Compiler内部型
意味領域 → Runtime
意味領域 → Backend
固定領域 → Source IR
固定領域 → Compiler実装
Runtime → SemanticModule
Backend → SemanticModule
Backend → ExecutionPlan
Backend → Shader source compiler
Frontend A → Frontend B
```

第12条 Visual Studio参照による強制

禁止依存は規約だけでなく、Visual StudioのProjectReferenceで物理的に防ぐ。

Backendプロジェクトから参照可能なのは原則として次だけとする。

```
Base
FrozenPackage
PackageRuntimeContract
Platform surface contract
D3D12固有utility
```

BackendからSemantic、Compiler、FrontendへProjectReferenceを張れない構造にする。

第5章 第二世代のプロジェクト構造

仮称として次の構成を採用する。

```
00_Base
01_MathVocabulary
02_SemanticModel
03_SemanticBuilder
04_SemanticAnalysis
05_TargetModel
06_D3D12TargetCompiler
07_FrozenPackage
08_PackageRuntime
09_D3D12Executor
```

10_PlatformWin32

20_ClassicalFrontend

21_SdfFrontend

22_PgaFrontend

23_ExperimentDomain

30_SemanticTests

31_CompilerTests

32_PackageContractTests

33_D3D12ConformanceTests

34_ExperimentTests

40_Launcher

各プロジェクトの責務

00_Base

StrongId

Result / Error

Stable integer types

Stable digest

Binary reader/writer

Checked arithmetic

Small utility

GPU概念を置かない。

01_MathVocabulary

Float2 / Float3 / Float4

Matrix

Camera specification

Transform

Experiment target

Classical、SDF、PGAのいずれにも属さない中立型だけを置く。

02_SemanticModel

純粋な意味グラフを定義する。

D3D12、Queue、Physical allocationを置かない。

03_SemanticBuilder

不正なSemantic graphを作りにくくする構築APIを提供する。

生のvectorへ直接pushする方式を正式APIにしない。

04_SemanticAnalysis

Validation
Dependency graph
Effect analysis
Range overlap
Temporal relation
Lifetime interval
Diagnostic source mapping

Target非依存の解析だけを行う。

05_TargetModel

Compilerが利用するTarget能力と方針を表す。

Queue topology
Memory classes
Frame policy
Shader target
Resource format support
Descriptor limits
Synchronization model

D3D12 C API型は含めない。

06_D3D12TargetCompiler

意味解析結果をD3D12実行向けに固定する。

Shader source compile、Reflection、binding layout決定もここに置く。

07_FrozenPackage

CompilerとBackendの国境。

Source IRへ依存しない。

filesystemへ依存しない。

D3D12 headerへ依存しない。

ただしD3D12 target packageであることを示すTarget kindやTarget-specific opaque dataは持てる。

08_PackageRuntime

LoadPackage
PackageInstance
FrameInvocation
FrameSubmission
Device epoch
External binding
Backend interface

SemanticModuleを知らない。

09_D3D12Executor

Frozen PackageをD3D12へ写像する。

Shader compileを行わない。

Schedulingを行わない。

State planningを行わない。

10_PlatformWin32

Window
Surface
Input events
Timing
Application loop

20以降のFrontend

各理論表現からSemanticModelへLoweringする。

相互に依存しない。

第6章 Semantic Model

第13条 Resource

Semantic Resourceは物理実装ではなく、論理的な情報領域を表す。

最低限、次を持つ。

```
ResourceId
Logical kind
Shape
Format meaning
Lifetime intent
Update intent
Visibility
Initial semantic contentの有無
```

`Persistent`、`Temporal`、`FrameLocal`、`External` は単なるBackend配置指定ではなく、時間的意味として定義する。

第14条 Resource Use

Resourceの利用情報を二重宣言してはならない。

第一世代のような、

```
Work.accesses
Payload.bindings
```

という二つの事実源を作らない。

正式構造は概念的に次とする。

```
ResourceUse
  Resource
  View
  Effect
  Role
  Temporal relation
```

Work payloadは `ResourceUseId` を参照する。

依存解析、状態解析、binding生成はすべて同じResourceUseから導出する。

第15条 Work

Workは「何を達成するか」を表す。

初期Work種別：

```
RasterWork
ComputeWork
```

CopyWork
PresentWork

ただしQueue種別は持たない。

QueueはCompilerが選ぶ。

第16条 Program Interface

Program Interfaceは意味的な入出力を一度だけ宣言する。

互換用の重複fieldを持たない。

暗黙のPOSITION/COLORデフォルトを持たない。

Program sourceとProgram interfaceを区別する。

```
ProgramSource
  HLSL source identityまたはsource bytes

ProgramInterface
  parameters
  vertex inputs
  outputs
  stage relations
```

SourceはCompiler入力であり、Frozen Packageには残さない。

第7章 Compiler pipeline

第17条 Compiler段階

Compilerは一つの巨大関数にしない。

最低限、次の段階に分ける。

1. Source validation
2. Canonical semantic graph
3. Effect and dependency analysis
4. Target feasibility
5. Scheduling
6. Resource instance planning
7. Allocation and alias planning
8. State-cell planning
9. Queue and synchronization planning

- 10. External boundary planning
- 11. Shader target compilation
- 12. Executable specialization
- 13. Operation lowering
- 14. Frozen Package validation
- 15. Serialization and digest

各段階は明示的な入力型と出力型を持つ。

第18条 Compilerの完全性

Backendで次の判断が必要になった場合、Compilerの仕事が未完成と判定する。

どのQueueを使うか
どのstateへ遷移するか
いつwaitするか
Resourceを何個作るか
どのResourceをaliasするか
いつ初期データをuploadするか
外部Resourceをいつ取得・返却するか
どのShaderをcompileするか
どのbinding layoutを使うか
いつPresentするか

第8章 Frozen Package契約

第19条 Package header

最低限、次を持つ。

Magic
Package format version
Target kind
Target profile digest
Package content digest
Required runtime version
Endianness
Flags
Section table

`size_t` を永続ABIへ使用しない。

固定幅整数を使用する。

第20条 Resource artifact

Resource artifactはSource declarationをコピーせず、実行に必要な情報だけを持つ。

- Resource identity
- Materialization kind
- Physical instance count
- Extent policy
- Format
- Usage requirements
- Allocation identity
- Initial state contract
- Initial content section
- Rebuild policy
- View descriptions

第21条 Program artifact

Program artifactは完全なCompiler成果物とする。

- Program identity
- Shader stage binaries
- Binary format
- Compiler target version
- Reflected interface
- Binding layout
- Root layout description
- Interface digest
- Debug source map (任意)

次を持つてはならない。

- filesystem::path
- HLSL source
- entry pointを再compileするための情報
- BackendがReflectionする必要がある未検証binary

第22条 Executable artifact

Executable artifactは、実行時にPSO等を生成するための完全な固定記述を持つ。

- Program artifact reference
- Vertex layout
- Attachment formats
- Raster state

```
Depth state
Blend state
Sample state
Specialization identity
```

BackendはExecutable artifactを再解釈して別の方針を選ばない。

第23条 Operation stream

Preparation、main execution、external boundary、Presentを一つの体系で表現する。

初期Operation候補：

```
CreatePackageResource
UploadBuffer
UploadTexture
InitializeState

AcquireExternal
WaitExternal

WaitQueue
WaitTemporal
BeginSubmission
ActivateAlias
Transition
RequireCommon
BindExecutable
ExecuteRaster
ExecuteCompute
ExecuteCopy
EndSubmission
SignalQueue

ReleaseExternal
PresentSurface
```

必ずしもすべてを一本のvectorへ平坦化する必要はない。

しかしBackend独自の隠れた実行順序を持たせてはならない。

Package上で順序関係が完全に決まっている必要がある。

第24条 External slot

External Resource slotは次を明示する。

```
Slot identity
Expected resource shape
Expected usage
Required incoming state
Guaranteed outgoing state
Acquire synchronization contract
Release synchronization contract
Required or optional
Device epoch compatibility
```

`Undefined` を「Backendが適当に推測せよ」という意味に使ってはならない。

状態が不明であること自体を許す場合は、明示的な `UnknownStateContract` として別の契約にする。

第25条 Package digest

Package digestはPackage内容全体から生成する。

最低限、次を含む。

```
Resource artifact
Initial content bytes
Shader binary bytes
Binding layout
Executable descriptions
Operation stream
External slot schema
Target profile
Package version
```

Debug nameやSource mapをdigestへ含めるかは、canonical digestとdebug digestを分けて決定する。

第9章 Runtime

第26条 Compileと実行の分離

正式APIは次の形とする。

```
CompileResult Compile(
    const SemanticGraph&,
    const TargetProfile&);

LoadedPackage LoadPackage(
    const FrozenExecutablePackage&);
```



```
FrameSubmission Submit(  
    LoadedPackage&,  
    const FrameInvocation&);
```

Core Runtimeに次のAPIを置かない。

```
Execute(const SemanticModule&);
```

便利な統合APIが必要なら、LauncherまたはExperiment用Facadeに置く。

第27条 Package instance

Serialized Packageと、Device上へ物質化されたPackage instanceを区別する。

```
FrozenExecutablePackage  
    不変なデータ  
  
LoadedPackage / PackageInstance  
    Device固有object  
    Descriptor  
    Resource  
    PS0  
    Fence tracking
```

Device loss時はFrozen PackageからPackageInstanceを再構築する。

第28条 FrameInvocation

FrameInvocationはPackageのInvocation schemaに対して検証される。

余分なResource、欠落Resource、サイズ違反、状態契約違反をRuntime境界で拒否する。

第10章 D3D12 Executor

第29条 Backendの許可事項

D3D12 Executorが決めてよいもの：

```
COM objectの生成手順  
D3D12 API呼び出し順への機械的変換  
Descriptor handleの実アドレス  
Fenceの実値  
Command allocatorの再利用
```

APIエラーの取得
DRED情報収集

第30条 Backendの禁止事項

D3D12 Executorが決めてはならないもの：

Workの並び替え
Queue選択
Resource寿命
Alias対象
Abstract state計画
External incoming/outgoing state
Shader compile option
Root layout方針
Presentの有無
Preparationの有無

第31条 Backend conformance

BackendはPackage Operationをすべて解釈できることをConformance testで証明する。

未対応Operationを黙って無視してはならない。

第11章 診断

第32条 二種類の診断

診断を分ける。

Compiler diagnostics
Source Resource / Work / Programへ対応
なぜ拒否または選択したかを説明

Runtime diagnostics
Package section / Operation indexへ対応
実行失敗、Device loss、External mismatchを説明

BackendはSource Workを直接知らない。

必要な場合はFrozen Package内の任意DebugMapを通じてSourceへ対応付ける。

第12章 テスト原則

第33条 テスト階層

テストを次に分離する。

```
Semantic tests
Compiler analysis tests
Package canonicalization tests
Serialization round-trip tests
Package corruption tests
Backend conformance tests
D3D12 WARP integration tests
Device removal tests
Frontend equivalence experiments
```

第34条 固定成果物の再現性

同一入力と同一Target profileから、byte単位で同じPackageを生成することを確認する。

単にhashが一致するだけでは不十分とする。

第35条 Resource upload

BufferとTextureの初期データはGPUへUpload後、Readbackして元bytesと一致することを確認する。

対象：

```
Buffer
Texture1D
Texture2D
Texture3D
Mip chain
Texture array
Row pitch差
Slice pitch差
```

Depth/stencil initial uploadを未対応とする場合は、すべてのDepth/stencil formatをCompilerで明示的に拒否する。

第36条 Device loss

次の流れを一続きで検証する。

Package load
Frame実行
Device removal検出
DRED記録
Device再作成
Package-owned object再物質化
Temporal history reset
External rebind要求
再実行

テスト用の単純なDevice再作成と、実際のDevice removed例外経路を分けて検証する。

第13章 最初の垂直スライス

第37条 初期機能

最初のスライスは次だけを扱う。

固定Triangle
単一Raster Work
単一Direct queue
単一Presentation surface
固定Vertex Buffer
固定Shader binary
1 frame in flight
No depth
No texture
No compute
No aliasing
No temporal
No external user resource

機能は小さくするが、境界は妥協しない。

必ず次の全経路を通す。

Classical triangle description
↓
Semantic Graph
↓
Analysis
↓
D3D12 Target Compiler
↓
Serialized Frozen Package

↓
Package Load
↓
D3D12 Executor
↓
Triangle表示

第38条 最初のスライスの完成条件

次をすべて満たした時だけ完成とする。

BackendがSemantic projectを参照していない
BackendがCompiler projectを参照していない
BackendがShader compilerを参照していない
Packageにfilesystem pathがない
PackageにShader sourceがない
Packageをファイルへ保存できる
別プロセスでPackageを読み込める
SourceなしでTriangleを表示できる
Package bytesが決定的
Package corruptionを検出できる
PresentがPackage operationとして表現される

第14章 その後の拡張順序

境界を壊さず、次の順序で機能を追加する。

Slice 1
Fixed Triangle

Slice 2
Dynamic constant slot
FrameInvocation

Slice 3
Buffer initial upload + Readback

Slice 4
Texture2D + Texture Readback

Slice 5
Depth attachment

Slice 6
Compute Work

Slice 7	Copy Work / Copy queue
Slice 8	Multiple queues / handoff
Slice 9	FrameLocal resource
Slice 10	Temporal resource
Slice 11	Aliasing
Slice 12	External resource Acquire / Release
Slice 13	Device loss and reconstruction
Slice 14	Classical / SDF common experiment
Slice 15	PGA direct frontend

後続Sliceのために、前段階で未使用の巨大な一般化を先に実装しない。

ただしFrozen Packageのversioningとsection構造は、最初から拡張可能に設計する。

第15章 設計変更手続き

第39条 憲法違反の扱い

実装上必要になったという理由だけで、禁止依存や責務境界を破ってはならない。

必要な情報が不足した場合は、次の順で対応する。

1. どの領域が情報を所有すべきか確認
2. 上流の契約へ情報を追加
3. Compilerで固定する
4. Package ABIをversion付きで拡張
5. Backendへ渡す

Backend内で一時的に推測して完成扱いにしてはならない。

第40条 変更記録

この文書を変更する場合は、次を記録する。

変更理由
破られた旧原則
新しい原則
代替案
長期的影響
Package ABIへの影響

最終原則

第二世代の中心はD3D12Backendではない。

Semantic Graphでもない。

中心は、

意味から導かれた判断を
欠落なく固定した
Frozen Executable Package

である。

自由領域は目的を記述する。

意味領域は論理を記述する。

Compiler領域は選択する。

固定領域は選択結果を保存する。

実行時領域は変化する値を与える。

Backendは保存された選択結果をD3D12へ写像する。

この境界を守る限り、将来PGA、SDF、CGA、Procedural geometry、あるいは現在存在しない数学的表現を追加しても、Backendを理論固有の都合で汚染せずに済む。

これを第二世代の出発点とする。